

Interactive Volume Rendering

Lee Westover

Abstract

Volume rendering - отображение данных в 3д пространстве. Однако для визуализации использовались линии или поверхности, которые необходимо было каким-то образом извлечь, что приводило к появлению артефактов

Introduction

Типичный тип данных для научных вычислений - регулярная 3д сетка выборочных точек. Прямое отображение (не переводя в поверхность или линии) volume (объемных) данных - volume render. Однако текущие технологии слишком медленные, так как рендереры трilinearно интерполируют точки вдоль каждого луча, или выполняют подгонку полигонов для каждой окрестности точки.

Но продвижения в машинной скорости и объеме памяти позволили перейти к прямому рендеру volume данных.

Есть два метода прямого рендера:

- 1) Прямое отображение (forward mapping) - метод volume rendering, при котором каждый воксель независимо проецируется на изображение и вносит вклад в пиксели экрана;
- 2) Обратное отображение (backward mapping) - луч выстреливается из каждого пикселя в объемные данные, пересекает, пока существуют данные или же луч не накопит достаточной прозрачности, чтобы стать непрозрачным.

DESIGN TRADEOFFS

Нужно разработать параллельную систему, но использование backward mapping - обратного отображения не подойдет, так как при распараллеливании каждый поток должен иметь доступ ко всем данным, так как луч пересекает их

Если же использовать forward mapping - прямое отображение, то мы разделяем объемные данные по узлам

ALGORITHM

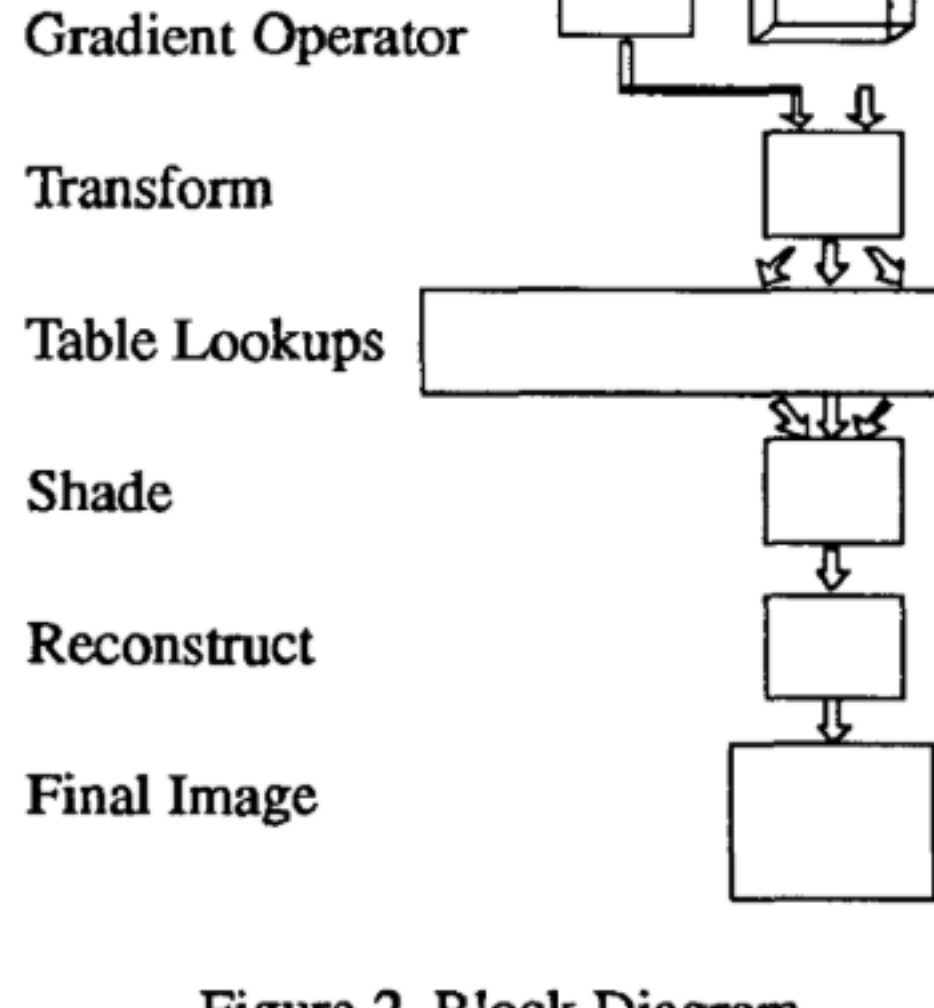


Figure 2. Block Diagram

1. Pipeline Structure

density value
gradient strength
gradient $\langle i, j, k \rangle$ direction
grid $\langle i, j, k \rangle$

density value - скаляр из исходного 3д массива

$gradient_i = data\langle i+1,j,k \rangle - data\langle i-1,j,k \rangle$
 $gradient_j = data\langle i,j+1,k \rangle - data\langle i,j-1,k \rangle$
 $gradient_k = data\langle i,j,k+1 \rangle - data\langle i,j,k-1 \rangle$

grid $\langle i,j,k \rangle$ - индексы в 3д массиве

Screen Mapping

density value
gradient strength
gradient $\langle i, j, k \rangle$ direction
screen $\langle x, y, z \rangle$

Shading, используем информацию о плотности и градиенте

red
green
blue
alpha
screen $\langle x, y, z \rangle$

Пик реконструкции и обхода в луче управления

когда все входные samples обработаны $\Rightarrow$ концы

Реконструкция после шейдинга в image space позволяет заменить дорогую 3D-интерполяцию и генерацию промежуточных samples на локальное 2D-распределение вклада уже зашедшего sample, что существенно снижает вычислительную стоимость и делает алгоритм интерактивным.

TRANSFORMATION AND RECONSTRUCTION:

1. Screen Mapping

$$\begin{bmatrix} \Delta x \\ \Delta y \\ \Delta z \end{bmatrix} = \begin{bmatrix} dx/di & dx/dj & dx/dk \\ dy/di & dy/dj & dy/dk \\ dz/di & dz/dj & dz/dk \end{bmatrix} \begin{bmatrix} \Delta i \\ \Delta j \\ \Delta k \end{bmatrix}$$

- приращения по алгоритму digital differential analyzer [DDA]

На вход - 3d grid. Нужно отобразить точку данных из пространства сетки в пространство экрана $\rightarrow$ используем DDA. В качестве начала для DDA выполняется полное матричное умножение матрицы преобразования (поворот + масштабирование). Потом просто добавляем delta.

При этом работаем только с ортогографической проекцией, так как в ней нет учета глубины, поэтому все шаги остаются постоянными

Заметка от AI:

Цель этапа: Эффективно преобразовать координаты каждого вокселя $\langle i, j, k \rangle$ из пространства регулярной 3D-сетки данных в экранное пространство $\langle x, y, z \rangle$ для последующей обработки.

Ключевые условия и ограничения:

Используется только ортогографическая проекция. Это упрощение, принятое для достижения скорости, так как при ортографии шаги преобразования постоянны.

Матрица преобразования включает только поворот и масштабирование (без перспективного искажения).

Механизм выполнения:

Инициализация: Для первой точки (начала обхода) выполняется полное матричное умножение для получения ее экранных координат $\langle x, y, z \rangle$. Эта точка становится началом для DDA.

Вычисление приращений (Deltas): Из матрицы преобразования напрямую извлекаются постоянные приращения $\Delta x, \Delta y, \Delta z$ для шагов вдоль осей сетки i, j, k .

Определение порядка обхода (Critical Point): Анализируя знак и величину Δz , рендерер определяет последовательность обхода вокселей, которая гарантирует их упорядоченность вдоль луча зрения. Это может быть порядок сзади-наперед (back-to-front) или спереди-назад (front-to-back). Выбор порядка влияет на последующую формулу композиции (combing).

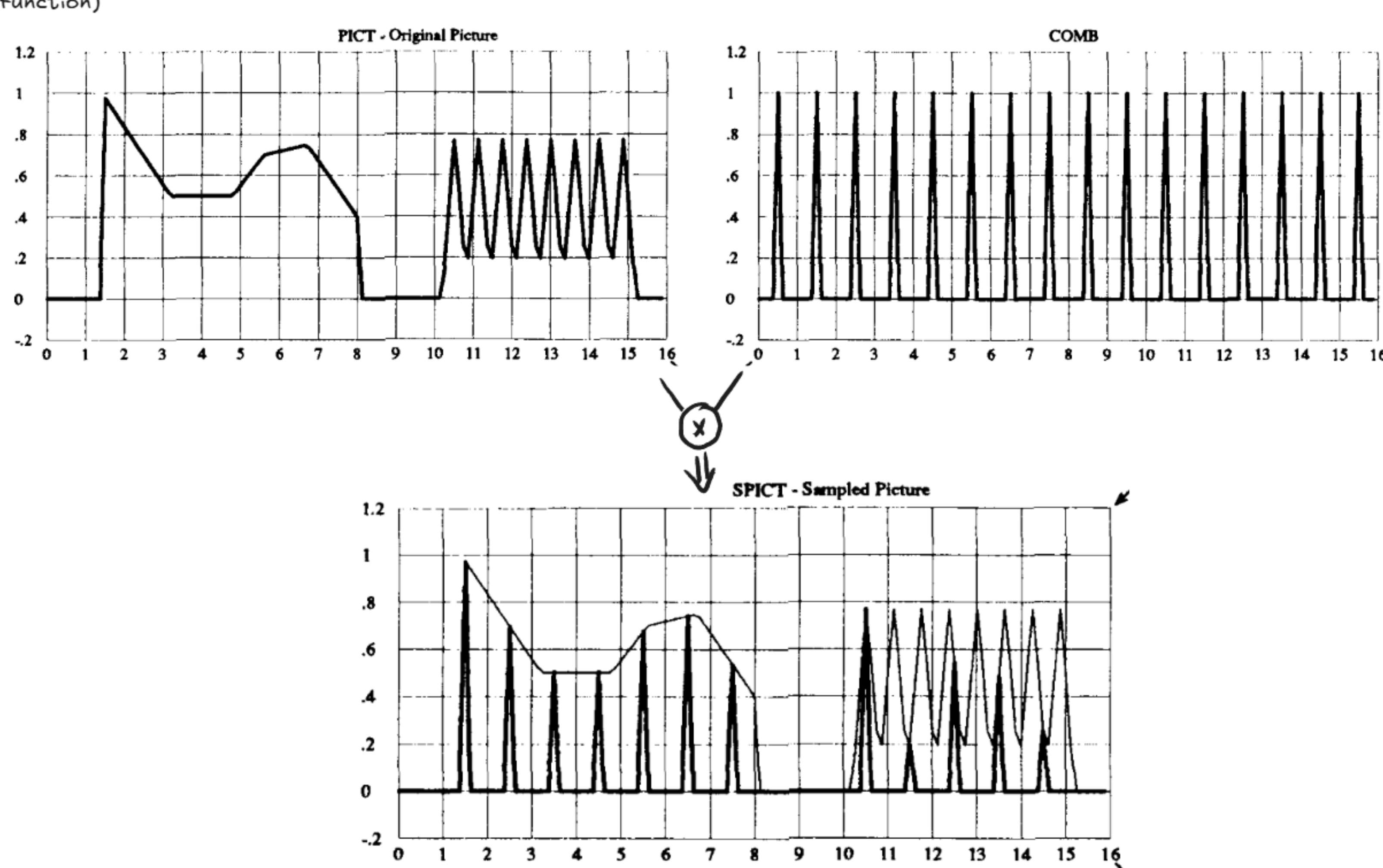
Инкрементальный обход: Рендерер проходит по всем вокселям сетки, добавляя соответствующие дельты $\Delta x, \Delta y, \Delta z$ к текущим экранным координатам. Это исключает необходимость в полном матричном умножении для каждой точки, обеспечивая высокую скорость.

Роль координаты Z в ортогографической проекции: Координата screen_z не является перспективной глубиной, а выступает как линейная мера расстояния от плоскости проекции. После вращения объема именно это значение позволяет сравнивать воксели и определять, какой из них "ближе" к наблюдателю для корректного расчета прозрачности.

Вывод: Этап Screen Mapping - это не просто смена системы координат, а создание оптимизированного, упорядоченного потока данных для конвейера, где z-упорядочение является фундаментальным для корректного объемного рендеринга.

2. Reconstruction

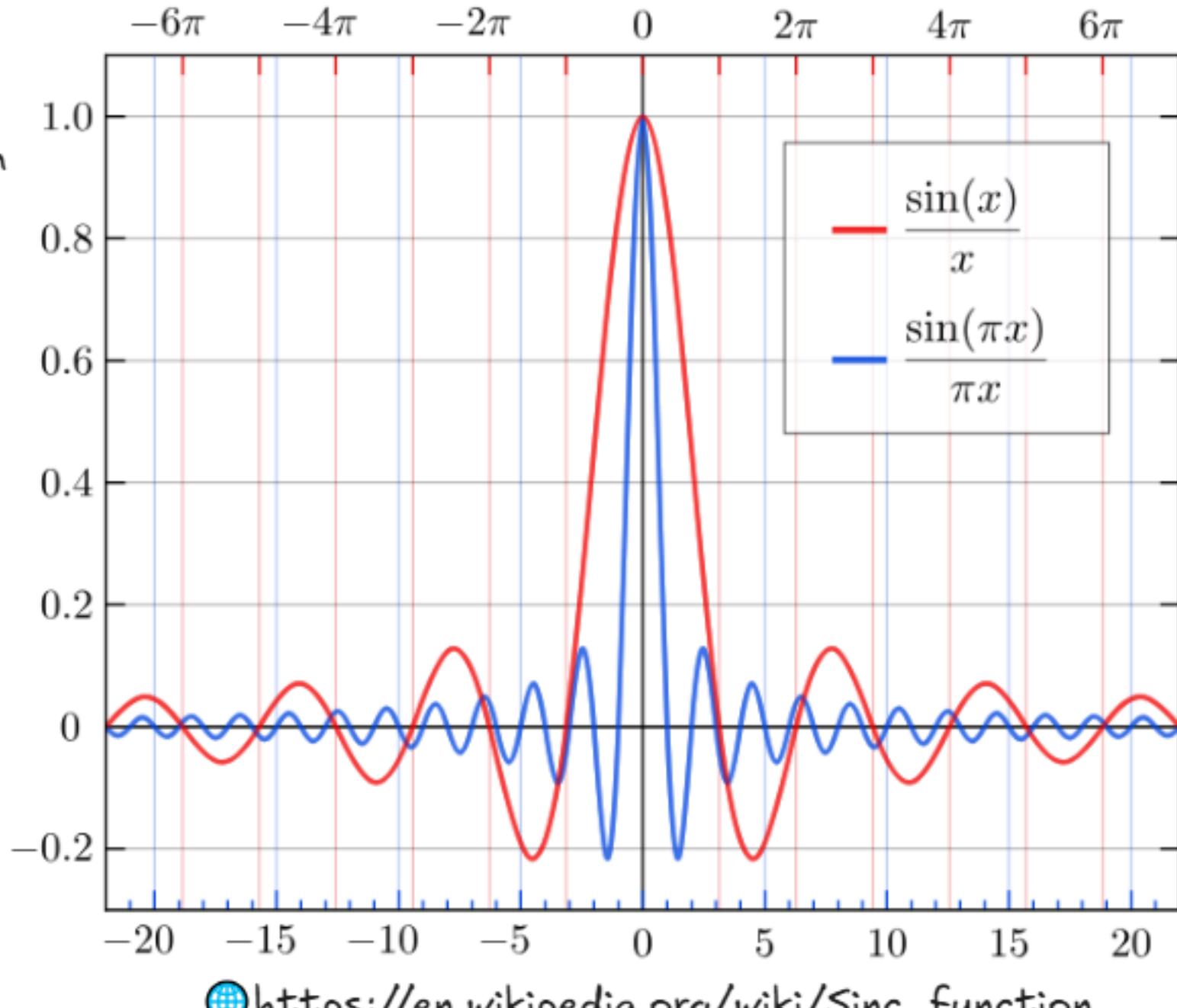
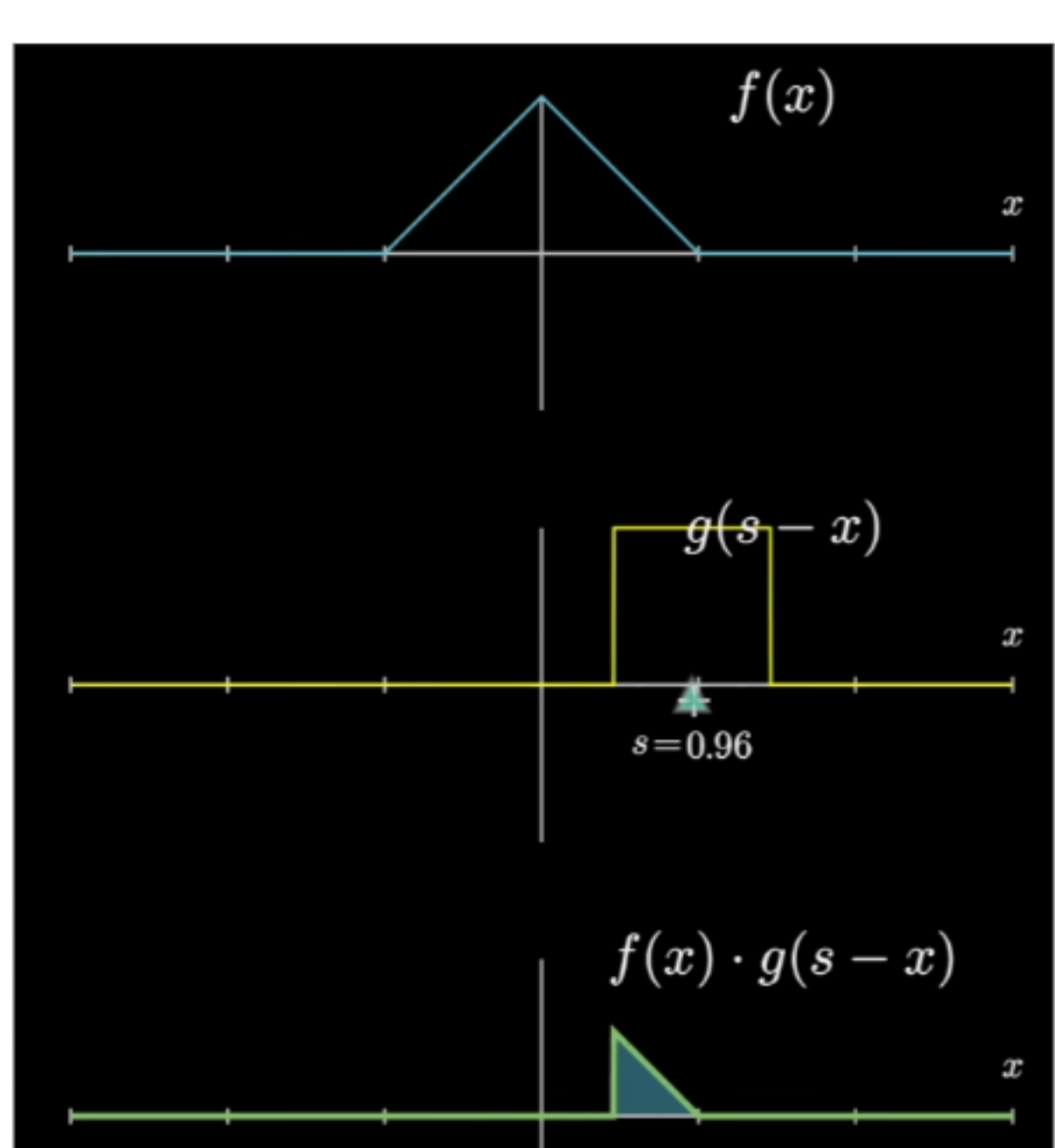
Так как у нас дискретный набор значений, то необходимо восстановить непрерывный сигнал. Сам выборочный сигнал можно рассмотреть как произведение непрерывного сигнала и функции гребенки (comb function)



Для восстановления можно использовать свертку выборочных данных с sinc функцией

<https://en.wikipedia.org/wiki/Convolution>

<https://youtu.be/IaSGqQa50-M?si=PkAMSzvHMU01GjT7>



$$[f * g](s) = \int_{-\infty}^{+\infty} f(x) g(s-x) dx$$

т.е. мы считаем произведение функций и потом интегрируем его $\Rightarrow$ получим функцию

Реконструкция:

$$signal_{3D} = \iiint_V h_V(u-x, v-y, w-z) \sum \delta(x, y, z) g(x, y, z) du dv dw$$

h_V - ядро реконструкции объема

g - ядро функции

$\sum \delta$ - comb (гребенка) ядро

Sinc-function

набор значений вокселей

гребенка-функция

$$\Rightarrow signal_{3D}(x, y, z) = \sum_{i \in Vol} h_V(i_x - x, i_y - y, i_z - z) g(i)$$

значения сетки в $\tau(x, y, z)$

увеличение в вокселях

вектор определений

каждый входной сигнал вносит

ка pixel (backward method)

в forward method отбрасывают

какие значения вносят

каждый sample

$\Rightarrow$ Чтобы того чтобы рассматривать, как несколько образцов влияют на одну точку, подумайте о том, как один образец может влиять на множество точек в пространстве. Влияние на точку $\langle x, y, z \rangle$ со стороны образца данных $\langle i \rangle$ выглядит так:

$$effect_i(x, y, z) = g(i) h_V(i_x - x, i_y - y, i_z - z)$$

$\Downarrow$

каждый воксель можно обраб отдельно

$\Downarrow$

наращением, единь

Combining:

$\Rightarrow$ проецируем на (x, y) по оси $z \Rightarrow$

$$effect_i(x, y) = \int_{-\infty}^{+\infty} h_V(i_x - x, i_y - y, w) g(i) dw$$

$\Downarrow$

$$effect_i(x, y) = g(i) \int_{-\infty}^{+\infty} h_V(i_x - x, i_y - y, w) dw$$

$\Downarrow$

интеграл не зависит от volume density ($g(i)$) $\Rightarrow$ можем вынести в

за скобки

$\Downarrow$

$$footprint(x, y) = \int_{-\infty}^{+\infty} h_V(x, y, w) dw$$

$\Downarrow$

определяем образ каждого pixel'a

$$weight(u, v)_p = footprint(p_x - u, p_y - v)$$

The process of table

$\Rightarrow$ lookup, weighting, and combining is called splatting.